

**Flutter como framework multiplataforma para el desarrollo móvil: análisis técnico y
comparación con Android nativo**

Kendall Chacón Molina

Diego Marín Lopez

Cristofer Zamora Arrieta

Andy Orozco Castro

Fernando Martinez

Universidad Nacional de Costa Rica

Diseño y programación de plataformas móviles

Rachel Bolívar Morales

6 de junio de 2026

Introducción

El desarrollo de aplicaciones móviles ha evolucionado debido a la necesidad de crear soluciones funcionales, escalables y disponibles en más de una plataforma. Tradicionalmente, las aplicaciones Android se han desarrollado mediante tecnologías nativas como Kotlin o Java, mientras que las aplicaciones iOS requieren herramientas propias del ecosistema Apple, como Swift. Este enfoque nativo ofrece integración profunda con cada sistema operativo, pero también puede implicar mayor tiempo de desarrollo, duplicación de código y equipos especializados para cada plataforma.

Ante este escenario, los frameworks multiplataforma han adquirido relevancia porque permiten construir aplicaciones para diferentes sistemas desde una base de código compartida. Flutter es un framework de código abierto desarrollado por Google que permite crear aplicaciones móviles, web, de escritorio y embebidas utilizando una sola base de código (Flutter, s. f.-f). Su propuesta técnica se basa en el uso del lenguaje Dart, un sistema de widgets propio, herramientas de desarrollo rápido y un motor de renderizado que le permite controlar directamente la forma en que se dibuja la interfaz.

El presente informe analiza Flutter desde una perspectiva técnica e investigativa. Para ello, se estudian su historia y origen, arquitectura, funcionamiento, lenguajes utilizados, casos reales de uso, comparación técnica con Android nativo, ventajas, desventajas, retos encontrados y análisis crítico del grupo. El propósito no es presentar Flutter como una solución universal, sino valorar en qué contextos representa una alternativa conveniente frente al desarrollo Android nativo y en cuáles sus limitaciones pueden afectar la decisión tecnológica.

Historia y origen de Flutter

Flutter fue desarrollado por Google como respuesta a la necesidad de construir interfaces de usuario modernas, rápidas y consistentes en diferentes plataformas. Antes de llamarse Flutter, el proyecto fue conocido como Sky, una propuesta experimental orientada

a ejecutar aplicaciones móviles con Dart y lograr alto rendimiento visual en Android. Amadeo (2015) señala que Sky buscaba explorar una alternativa al modelo tradicional de desarrollo móvil basado en Java, con la promesa de interfaces rápidas y una experiencia visual fluida.

La evolución de Flutter se relaciona con una decisión técnica importante: no depender por completo de los componentes visuales nativos del sistema operativo. En lugar de construir la interfaz con vistas nativas de Android o iOS, Flutter utiliza su propio sistema de widgets y su propio motor de renderizado. Esta característica permitió que el framework se orientara hacia una experiencia visual consistente entre plataformas, reduciendo las diferencias que normalmente aparecen cuando se mantienen aplicaciones separadas para Android e iOS.

Un punto clave en su historia ocurrió el 4 de diciembre de 2018, cuando Google anunció Flutter 1.0 como la primera versión estable del framework. En esa publicación, Sneath (2018) presentó Flutter como un toolkit de interfaz de usuario para crear experiencias nativas en iOS y Android desde una sola base de código. Posteriormente, con Flutter 2, Google fortaleció la visión multiplataforma al anunciar soporte para Android, iOS, web, Windows, macOS y Linux desde una misma base de código (Google Developers Blog, 2021).

En síntesis, Flutter surgió como un experimento técnico dentro de Google, pero evolucionó hasta convertirse en una tecnología multiplataforma utilizada en proyectos profesionales. Su historia muestra una transición desde el desarrollo móvil hacia una visión más amplia, donde una sola base de código puede servir para crear aplicaciones en distintos dispositivos y sistemas operativos.

Arquitectura y funcionamiento

Flutter posee una arquitectura basada en capas. De acuerdo con su documentación oficial, el framework se organiza alrededor de varios niveles: la aplicación escrita en Dart, el

framework de Flutter, el motor gráfico y el embedder de la plataforma (Flutter, s. f.-b). Esta estructura permite que una aplicación Flutter pueda ejecutarse sobre diferentes sistemas operativos manteniendo una parte importante del código compartido.

La capa superior corresponde a la aplicación escrita en Dart. En esta sección se encuentran las pantallas, la lógica de presentación, los widgets, los modelos y parte de las reglas de negocio. Debajo se ubica el framework de Flutter, que proporciona bibliotecas para construir interfaces, gestionar animaciones, manejar gestos, aplicar accesibilidad, trabajar con texto, organizar navegación y definir componentes visuales. El motor, escrito principalmente en C++, se encarga de tareas de bajo nivel como renderizado gráfico, composición visual, procesamiento de texto y ejecución de Dart.

Uno de los conceptos centrales de Flutter es que la interfaz se construye mediante widgets. Un widget puede representar un texto, un botón, una imagen, una fila, una columna, un margen, una tarjeta o incluso una pantalla completa. Estos widgets se organizan en forma de árbol, donde cada elemento puede contener otros widgets hijos. Esta forma de trabajo favorece la composición de interfaces complejas a partir de componentes pequeños y reutilizables.

Flutter utiliza un modelo declarativo. En lugar de modificar manualmente cada elemento visual, el desarrollador describe cómo debe verse la interfaz según el estado actual de la aplicación. Cuando el estado cambia, Flutter reconstruye las partes necesarias del árbol de widgets. Para componentes que no cambian internamente se utiliza `StatelessWidget`, mientras que para componentes que pueden actualizarse durante la ejecución se utiliza `StatefulWidget` u otros mecanismos de manejo de estado.

El renderizado es una diferencia importante frente al desarrollo nativo tradicional. Flutter no depende completamente de los componentes visuales nativos de Android o iOS, sino que dibuja su propia interfaz. En versiones recientes, Flutter utiliza Impeller como motor de renderizado en plataformas compatibles. Impeller precompila shaders durante la construcción del motor para evitar compilaciones en tiempo de ejecución y reducir problemas visuales como el jank (Flutter, s. f.-h).

Lenguajes utilizados

El lenguaje principal utilizado por Flutter es Dart. Dart es un lenguaje desarrollado por Google, de código abierto, orientado a objetos y diseñado para construir aplicaciones de alta calidad en diferentes plataformas (Dart, s. f.-a). Su integración con Flutter es directa, ya que las pantallas, widgets, lógica de presentación y comportamiento de la aplicación se escriben principalmente en Dart.

Dart ofrece características importantes para el desarrollo móvil moderno. Entre ellas se encuentran el tipado estático, la inferencia de tipos, la seguridad frente a valores nulos, la programación orientada a objetos, el uso de clases y la asincronía mediante Future, async y await (Dart, s. f.-b). Estas características son relevantes porque una aplicación móvil normalmente debe consumir servicios externos, procesar datos, manejar formularios, guardar información localmente y ejecutar tareas sin bloquear la interfaz de usuario.

Aunque Dart es el lenguaje principal de Flutter, no es el único lenguaje que puede intervenir en un proyecto. Cuando se necesita acceder a funciones específicas de la plataforma, Flutter permite escribir código nativo mediante platform channels. Este mecanismo permite comunicar el código Dart con Kotlin o Java en Android, y con Swift u Objective-C en iOS (Flutter, s. f.-i). Por esta razón, Flutter no elimina por completo la necesidad de conocer tecnologías nativas, especialmente en aplicaciones que requieren integración profunda con sensores, servicios del sistema, permisos específicos o APIs no cubiertas por paquetes existentes.

Casos reales de uso

Flutter se utiliza en aplicaciones reales de empresas internacionales que requieren rendimiento, mantenimiento continuo y despliegue multiplataforma. Estos casos permiten observar que el framework no se limita al ámbito académico o experimental, sino que ha sido adoptado en productos con usuarios reales y necesidades empresariales complejas.

Uno de los casos más importantes es Google Pay. Según el caso oficial publicado por Flutter, Google Pay tenía 1.7 millones de líneas de código entre sus aplicaciones Android e iOS, situación que dificultaba la expansión a nuevos países y obligaba a duplicar funcionalidades. Con Flutter, el equipo logró una reducción del 70 % en esfuerzo de ingeniería y una reducción del 35 % en líneas de código (Flutter, s. f.-g). Este caso evidencia una de las ventajas más fuertes del framework: reducir duplicación cuando una aplicación debe mantenerse simultáneamente en Android e iOS.

Otro caso relevante es BMW. La empresa enfrentaba diferencias entre sus aplicaciones iOS y Android, además de la complejidad de manejar variantes por países, marcas y regulaciones. Flutter fue utilizado para construir la aplicación My BMW con una base común que permitiera mantener consistencia funcional y visual entre plataformas. Según el caso oficial, la aplicación fue lanzada en julio de 2020 y se consolidó en 47 países como una interfaz entre el usuario, el vehículo y los servicios digitales de la marca (Flutter, s. f.-c).

eBay Motors también representa un caso significativo. El equipo necesitaba crear una aplicación para Android e iOS en menos de un año, con un equipo limitado y una experiencia visual consistente. Flutter permitió compartir el 98.3 % del código y lanzar nuevas versiones en ambas tiendas con mayor frecuencia (Flutter, s. f.-d). Alibaba Group, mediante la aplicación Xianyu, utilizó Flutter para mejorar la experiencia de usuario en un mercado de segunda mano en China, con una interfaz consistente entre plataformas (Flutter, s. f.-a).

Comparación técnica con Android nativo

Flutter y Android nativo representan dos enfoques distintos para desarrollar aplicaciones móviles. Android nativo se orienta específicamente al ecosistema Android y utiliza principalmente Kotlin o Java. Flutter, en cambio, utiliza Dart y permite construir aplicaciones para Android, iOS, web y escritorio desde una misma base de código.

Desde el punto de vista arquitectónico, Android nativo trabaja directamente sobre los componentes del sistema operativo. La documentación oficial de Android recomienda diseñar aplicaciones con al menos dos capas: una capa de interfaz de usuario y una capa de datos. Además, puede agregarse una capa de dominio para simplificar y reutilizar interacciones entre ambas (Android Developers, s. f.-a). Este enfoque favorece la separación de responsabilidades, la escalabilidad, la mantenibilidad y la prueba de componentes.

Flutter también puede organizarse con arquitectura por capas, pero su estructura se basa en widgets, estado, motor de renderizado y embedder. Mientras Android nativo se integra directamente con el sistema operativo, Flutter utiliza una capa intermedia que le permite ejecutar la misma aplicación en diferentes plataformas. Esta diferencia tiene consecuencias importantes: Android nativo ofrece acceso más directo a las APIs del sistema, mientras que Flutter facilita la reutilización multiplataforma.

En cuanto a la interfaz gráfica, Android nativo puede utilizar vistas XML tradicionales o Jetpack Compose. Jetpack Compose es el toolkit moderno recomendado por Android para construir interfaces nativas con Kotlin. Su modelo declarativo permite describir la interfaz y actualizarla automáticamente cuando cambia el estado de la aplicación (Android Developers, s. f.-b). Flutter utiliza una lógica similar en cuanto a declaración de interfaz, pero la implementa mediante widgets propios y un motor gráfico independiente.

En términos de rendimiento, Android nativo puede tener ventaja cuando la aplicación depende intensivamente de características específicas del sistema operativo, servicios en segundo plano, APIs recientes o integración directa con hardware. Flutter puede alcanzar un rendimiento adecuado, pero requiere aplicar buenas prácticas de construcción de widgets, manejo de imágenes, animaciones y estado. La documentación de Flutter recomienda medir el rendimiento para evitar jank o interrupciones visuales (Flutter, s. f.-e).

Tabla 1

Comparación técnica entre Flutter y Android nativo

Criterio	Flutter	Android nativo
Lenguaje	Dart, con apoyo nativo opcional.	Kotlin o Java.
Interfaz	Widgets propios y motor de renderizado.	XML o Jetpack Compose integrado al sistema.
Plataformas	Android, iOS, web y escritorio.	Principalmente Android.
Mantenimiento	Mayor reutilización de código.	Más control nativo, pero menor reutilización si se requiere iOS.
Integración	Plugins y platform channels.	Acceso directo al SDK de Android.
Caso recomendado	Productos multiplataforma con identidad visual común.	Apps con integración profunda al ecosistema Android.

En mantenimiento, Flutter presenta una ventaja clara cuando el proyecto debe publicarse en Android e iOS. Una sola base de código puede reducir duplicación de pantallas, validaciones, modelos, lógica de presentación y pruebas. En Android nativo, si también se necesita una versión para iOS, normalmente se debe mantener otro proyecto separado con Swift, lo que aumenta el esfuerzo de coordinación y puede generar diferencias entre plataformas.

Por tanto, Flutter no reemplaza completamente a Android nativo. Flutter es más conveniente cuando se busca productividad multiplataforma, consistencia visual y reducción de duplicación. Android nativo es preferible cuando la aplicación requiere integración profunda con Android, máximo control sobre APIs del sistema o una experiencia completamente ajustada a las convenciones nativas de la plataforma.

Ventajas de Flutter

La principal ventaja de Flutter es la posibilidad de construir aplicaciones para varias plataformas desde una sola base de código. Esto permite reducir duplicación, acelerar entregas y mantener una experiencia más consistente entre Android e iOS. Para equipos pequeños o proyectos con plazos ajustados, esta característica puede representar una diferencia significativa.

Otra ventaja importante es la productividad durante el desarrollo. Flutter ofrece herramientas como hot reload, que permite aplicar cambios de forma rápida durante la ejecución de la aplicación. Esta funcionalidad facilita corregir errores visuales, ajustar interfaces, probar comportamientos y mejorar la experiencia de desarrollo.

Flutter también destaca por su sistema de widgets. Al construir la interfaz mediante componentes reutilizables, el equipo puede crear una biblioteca visual consistente y mantener una estructura más ordenada. Este modelo favorece la componentización, la reutilización y la creación de pantallas complejas a partir de piezas pequeñas.

La consistencia visual entre plataformas también es una ventaja relevante. Como Flutter controla su propio renderizado, puede mantener una apariencia similar en Android e iOS. Esto resulta útil para productos que necesitan conservar una identidad visual fuerte, independientemente del dispositivo utilizado por el usuario.

Desventajas y limitaciones de Flutter

Una de las principales desventajas de Flutter es el tamaño inicial de las aplicaciones. Debido a que incluye su propio motor de renderizado y recursos necesarios para ejecutar la interfaz, una aplicación Flutter puede tener un tamaño base mayor que una aplicación nativa simple. Aunque existen técnicas para optimizar el tamaño, este aspecto debe considerarse especialmente en proyectos donde el peso de descarga es crítico.

Otra limitación es la dependencia del ecosistema de paquetes. Muchas funcionalidades se resuelven mediante plugins, pero no todos los paquetes son igual de confiables, actualizados o compatibles con versiones recientes del framework. Una mala elección de dependencias puede provocar problemas de mantenimiento, errores de compatibilidad o bloqueos en futuras actualizaciones.

La curva de aprendizaje también puede ser una dificultad. El equipo debe aprender Dart, el sistema de widgets, el ciclo de vida de los componentes, la navegación, el manejo de estado y la estructura propia de Flutter. Para desarrolladores con experiencia en Android

nativo, el cambio puede requerir adaptación porque Flutter no trabaja exactamente igual que Activities, Fragments, XML o incluso Jetpack Compose.

Otra limitación se relaciona con la integración nativa. Aunque Flutter permite acceder a funcionalidades específicas mediante platform channels, esta solución exige conocimiento adicional de Kotlin, Java, Swift u Objective-C. Por tanto, en aplicaciones que dependen mucho de hardware, sensores, servicios en segundo plano o APIs específicas, Flutter puede requerir trabajo adicional.

El rendimiento también puede convertirse en un problema si no se aplican buenas prácticas. Flutter puede ofrecer interfaces fluidas, pero una mala gestión del estado, imágenes pesadas, reconstrucciones innecesarias o layouts complejos pueden generar pérdida de rendimiento. Por eso, el equipo debe medir y optimizar, no asumir que la aplicación será eficiente automáticamente.

Retos encontrados

El primer reto al trabajar con Flutter es comprender su modelo declarativo. Para equipos acostumbrados al desarrollo Android tradicional, puede ser difícil pasar de una lógica basada en vistas, actividades o fragmentos a una estructura basada en árboles de widgets y reconstrucción de interfaz según el estado.

El segundo reto es el manejo de estado. En aplicaciones pequeñas, setState puede ser suficiente, pero en proyectos reales se requiere una estrategia más organizada. Si el estado no se administra correctamente, el código puede volverse difícil de mantener, probar y escalar.

El tercer reto es la selección de dependencias. Flutter cuenta con un ecosistema amplio, pero cada paquete debe evaluarse antes de incorporarse al proyecto. Es necesario revisar documentación, mantenimiento, compatibilidad, cantidad de descargas, historial de actualizaciones y soporte de la comunidad.

El cuarto reto es la integración con funcionalidades nativas. Aunque muchos plugins resuelven tareas comunes, algunas características específicas pueden requerir código nativo. Esto obliga a que el equipo no dependa únicamente de Dart, sino que conserve conocimientos básicos de Android e iOS.

El quinto reto es la optimización del rendimiento y la adaptación de experiencia de usuario. Una aplicación multiplataforma no debe limitarse a verse igual en todos los dispositivos; también debe sentirse adecuada en cada sistema operativo. Por ello, es necesario adaptar gestos, navegación, permisos, accesibilidad y patrones visuales según la plataforma.

Análisis crítico del grupo

Como grupo, consideramos que Flutter es una alternativa sólida y madura para el desarrollo móvil multiplataforma. Su principal valor se encuentra en la posibilidad de construir aplicaciones para Android e iOS desde una base de código compartida, reduciendo duplicación y facilitando el mantenimiento. Además, su sistema de widgets y herramientas como hot reload permiten mejorar la productividad del equipo y acelerar la construcción de interfaces.

No obstante, Flutter no debe asumirse como la mejor opción para todos los proyectos. Su adopción debe responder a una evaluación técnica. Si una aplicación requiere una fuerte integración con características específicas de Android, uso intensivo de sensores, servicios en segundo plano complejos o acceso inmediato a APIs recientes del sistema operativo, Android nativo puede ser una opción más adecuada.

Desde nuestra perspectiva, Flutter resulta especialmente conveniente en aplicaciones comerciales, educativas, informativas, de comercio electrónico, formularios, gestión interna o productos que necesitan llegar tanto a Android como a iOS con recursos limitados. En estos casos, la reutilización de código, la consistencia visual y la velocidad de desarrollo pueden justificar su adopción.

El criterio final del grupo es que Flutter no reemplaza al desarrollo Android nativo, sino que amplía las posibilidades de decisión tecnológica. Es una herramienta recomendable cuando el objetivo principal es desarrollar de forma eficiente para varias plataformas. No obstante, su uso exige disciplina técnica: arquitectura clara, manejo adecuado del estado, selección responsable de paquetes, pruebas, monitoreo de rendimiento y conocimiento básico de integración nativa.

Conclusión

Flutter ha evolucionado desde un proyecto experimental de Google hasta convertirse en un framework multiplataforma utilizado en aplicaciones reales de empresas reconocidas. Su arquitectura basada en widgets, su motor de renderizado, el uso de Dart y su capacidad para compartir código entre plataformas lo convierten en una opción atractiva para el desarrollo móvil moderno.

En comparación con Android nativo, Flutter destaca por productividad, reutilización de código y consistencia visual. Sin embargo, Android nativo mantiene ventajas en integración directa con el sistema operativo, acceso a APIs específicas, rendimiento en escenarios altamente dependientes del hardware y adaptación completa a los patrones propios de Android.

Por tanto, la elección entre Flutter y Android nativo debe depender del tipo de proyecto, las plataformas objetivo, el equipo disponible, los plazos de desarrollo y el nivel de integración requerido. Flutter es recomendable cuando se busca eficiencia multiplataforma; Android nativo es preferible cuando se necesita máximo control sobre el ecosistema Android.+

Referencias

- Amadeo, R. (2015, 1 de mayo). *Google's Dart language on Android aims for Java-free, 120 FPS apps.* Ars Technica. <https://arstechnica.com/gadgets/2015/05/googles-dart-language-on-android-aims-for-java-free-120-fps-apps/>
- Android Developers. (s. f.-a). *Guide to app architecture*. Recuperado el 3 de junio de 2026, de <https://developer.android.com/topic/architecture>
- Android Developers. (s. f.-b). *Jetpack Compose*. Recuperado el 3 de junio de 2026, de <https://developer.android.com/compose>
- Dart. (s. f.-a). *Dart programming language*. Recuperado el 3 de junio de 2026, de <https://dart.dev/>
- Dart. (s. f.-b). *Introduction to Dart*. Recuperado el 3 de junio de 2026, de <https://dart.dev/language>
- Flutter. (s. f.-a). *Alibaba Group*. Recuperado el 3 de junio de 2026, de <https://flutter.dev/showcase/alibaba-group>
- Flutter. (s. f.-b). *Architectural overview*. Recuperado el 3 de junio de 2026, de <https://docs.flutter.dev/resources/architectural-overview>
- Flutter. (s. f.-c). *BMW*. Recuperado el 3 de junio de 2026, de <https://flutter.dev/showcase/bmw>
- Flutter. (s. f.-d). *eBay*. Recuperado el 3 de junio de 2026, de <https://flutter.dev/showcase/ebay>
- Flutter. (s. f.-e). *Flutter performance profiling*. Recuperado el 3 de junio de 2026, de <https://docs.flutter.dev/perf/ui-performance>
- Flutter. (s. f.-f). *Flutter: Build apps for any screen*. Recuperado el 3 de junio de 2026, de <https://flutter.dev/>
- Flutter. (s. f.-g). *Google Pay*. Recuperado el 3 de junio de 2026, de <https://flutter.dev/showcase/google-pay>

Flutter. (s. f.-h). *Impeller rendering engine*. Recuperado el 3 de junio de 2026, de <https://docs.flutter.dev/perf/impeller>

Flutter. (s. f.-i). *Writing custom platform-specific code*. Recuperado el 3 de junio de 2026, de <https://docs.flutter.dev/platform-integration/platform-channels>

Google Developers Blog. (2021, 3 de marzo). *Announcing Flutter 2*. <https://developers.googleblog.com/announcing-flutter-2/>

Sneath, T. (2018, 4 de diciembre). *Flutter 1.0: Google's portable UI toolkit*. Google Developers Blog. <https://developers.googleblog.com/flutter-10-googles-portable-ui-toolkit/>